机器人数值优化算法作业

目录

1 作业说明 **3**

2 线性最小二乘 **4**

2.1 题目 1：三种方法求解线性最小二乘 4

2.2 题目 2：病态最小二乘问题 4

3 非线性最小二乘与 Gauss–Newton **6**

3.1 题目 3：实现一维 Gauss–Newton 6

3.2 题目 4：二维 UWB 定位 7

4 Newton 与 Gauss–Newton **8**

4.1 题目 5：Newton 与 GN 的 Hessian 对比 8

5 Levenberg–Marquardt **9**

5.1 题目 6：实现完整 LM 9

5.2 题目 7：研究 λ 对 LM 更新方向的影响 10

6 Line Search **11**

6.1 题目 8：实现 Backtracking Line Search 11

6.2 题目 9：Gauss–Newton + Line Search 12

7 Trust Region **13**

7.1 题目 10：二维 Trust Region 实验 13

8 BFGS **14**

8.1 题目 11：从零实现 BFGS 14

9 L-BFGS **16**

9.1 题目 12：实现 Two-loop Recursion 16

10 机器人轨迹估计综合实验 **17**

10.1 题目 13：二维机器人轨迹平滑 17

11 可观测性与退化 **19**

11.1 题目 14：UWB 几何退化实验 19

12 综合大作业 **20**

12.1 题目 15：实现一个二维机器人非线性优化实验框架 20

13 提高题 **21**

13.1 题目 16：加权 UWB 最小二乘 21

13.2 题目 17：异常值与鲁棒核函数 22

14 建议评分 **22**

15 建议完成顺序 **23**

1 作业说明

本作业主要考察机器人数值优化中的基本算法及其实际实现，包括：

- 线性最小二乘；
- 非线性最小二乘；
- Newton 方法；
- Gauss–Newton 方法；
- Levenberg–Marquardt 方法；
- Line Search；
- Trust Region；
- BFGS；
- L-BFGS；
- 数值稳定性、条件数和可观测性分析。

除特别说明外，编程题建议使用 Python + NumPy + Matplotlib 完成。

对于要求实现优化算法的题目，原则上不得直接调用 `scipy.optimize.least_squares`、`scipy.optimize.minimize` 等高级优化器完成核心算法。

允许使用：

- `numpy.linalg.solve`；
- `numpy.linalg.qr`；
- `numpy.linalg.svd`；
- `numpy.linalg.cholesky`；
- Matplotlib 进行可视化。

建议每个实验至少提交：

1. 数学模型；
2. 核心代码；
3. 运行结果；
4. 可视化图像；
5. 对算法行为的分析。

2 线性最小二乘

2.1 题目 1：三种方法求解线性最小二乘

给定矩阵

$$A = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \\ 4 & 1 \\ 5 & 1 \end{bmatrix}, \qquad \boldsymbol{b} = \begin{bmatrix} 2.2 \\ 2.8 \\ 4.1 \\ 5.1 \\ 5.9 \end{bmatrix}.$$

考虑线性模型

$$y = ax + b.$$

令

$$\boldsymbol{x} = \begin{bmatrix} a \\ b \end{bmatrix}.$$

完成以下任务：

(1) 推导正规方程

$$A^T A\boldsymbol{x} = A^T \boldsymbol{b}.$$

(2) 使用以下三种方式分别求解：

(a) 正规方程；

(b) QR 分解；

(c) SVD。

(3) 输出三种方法得到的参数 a, b。

(4) 绘制原始数据点和三种方法得到的拟合直线。

(5) 比较三种结果之间的数值误差。

2.2 题目 2：病态最小二乘问题

构造矩阵

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1+\epsilon \\ 1 & 1+2\epsilon \\ 1 & 1+3\epsilon \end{bmatrix},$$

其中

$$\epsilon = 10^{-1}, 10^{-3}, 10^{-5}, 10^{-7}.$$

随机生成真实参数

$$\boldsymbol{x}_{\text{true}} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

并构造：

$$\boldsymbol{b} = A\boldsymbol{x}_{\text{true}} + \boldsymbol{\epsilon},$$

其中测量噪声服从：

$$\boldsymbol{\epsilon} \sim \mathcal{N}(0, 10^{-6} I).$$

完成：

(1) 对每一个 ϵ 计算

$$\kappa(A), \qquad \kappa(A^T A).$$

(2) 验证数值上是否近似满足

$$\kappa(A^T A) \approx \kappa(A)^2.$$

(3) 分别使用 Normal Equation、QR、SVD 求解 $\boldsymbol{x}$。

(4) 绘制横轴为 ϵ、纵轴为

$$\|\boldsymbol{x}_{\text{est}} - \boldsymbol{x}_{\text{true}}\|$$

的对数图。

(5) 分析为什么正规方程在病态问题上更容易出现数值误差。

3 非线性最小二乘与 Gauss–Newton

3.1 题目 3：实现一维 Gauss–Newton

考虑模型

$$y = ae^{bx}.$$

生成数据：

$$x_i \in [0, 2],$$

真实参数：

$$a = 2, \qquad b = -1.5,$$

并加入高斯噪声：

$$y_i = 2e^{-1.5x_i} + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, 0.05^2).$$

参数向量：

$$\boldsymbol{\theta} = \begin{bmatrix} a \\ b \end{bmatrix}.$$

残差定义为：

$$r_i(\boldsymbol{\theta}) = ae^{bx_i} - y_i.$$

完成：

(1) 手工推导 Jacobian：

$$\frac{\partial r_i}{\partial a}, \qquad \frac{\partial r_i}{\partial b}.$$

(2) 自己实现 Gauss–Newton：

$$J^T J \Delta\boldsymbol{\theta} = -J^T \boldsymbol{r}.$$

(3) 使用初值

$$\boldsymbol{\theta}_0 = \begin{bmatrix} 1 \\ -0.2 \end{bmatrix}.$$

(4) 每一轮记录：

$$F_k = \frac{1}{2}\|\boldsymbol{r}_k\|^2.$$

(5) 绘制：

(a) 原始数据和最终拟合曲线；

(b) cost–iteration 曲线；

(c) 参数 a, b 的迭代过程。

3.2 题目 4：二维 UWB 定位

二维平面上有四个 UWB 基站：

$$\boldsymbol{p}_1 = (0,0), \qquad \boldsymbol{p}_2 = (10,0), \qquad \boldsymbol{p}_3 = (10,10), \qquad \boldsymbol{p}_4 = (0,10).$$

随机生成机器人真实位置：

$$\boldsymbol{x}_{\text{true}} \in [2,8] \times [2,8].$$

测距数据：

$$d_i = \|\boldsymbol{x}_{\text{true}} - \boldsymbol{p}_i\| + \epsilon_i,$$

其中：

$$\epsilon_i \sim \mathcal{N}(0, 0.1^2).$$

定义残差：

$$r_i(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{p}_i\| - d_i.$$

完成：

(1) 推导 Jacobian：

$$J_i = \frac{\partial r_i}{\partial \boldsymbol{x}}.$$

(2) 从零实现 Gauss–Newton 定位器。

(3) 使用有限差分：

$$\frac{\partial r}{\partial x_j} \approx \frac{r(\boldsymbol{x} + \epsilon \boldsymbol{e}_j) - r(\boldsymbol{x} - \epsilon \boldsymbol{e}_j)}{2\epsilon}$$

验证解析 Jacobian。

(4) 绘制：

(a) UWB 基站；

(b) UWB 测距圆；

(c) 真实位置；

(d) 初始位置；

(e) GN 每一步优化轨迹。

(5) 在二维空间上计算

$$F(x,y)=\frac{1}{2}\sum_i r_i(x,y)^2$$

并绘制 cost 等高线。

(6) 将 GN 轨迹叠加到等高线上。

(7) 分析为什么 GN 的更新方向一般不垂直于等高线。

4 Newton 与 Gauss–Newton

4.1 题目 5：Newton 与 GN 的 Hessian 对比

继续使用题目 4 的 UWB 模型。

目标函数：

$$F(\boldsymbol{x})=\frac{1}{2}\sum_i r_i(\boldsymbol{x})^2.$$

完成：

(1) 推导真实 Hessian：

$$\nabla^2 F=J^TJ+\sum_i r_i\nabla^2 r_i.$$

(2) 推导 UWB 距离残差

$$r_i(\boldsymbol{x})=\|\boldsymbol{x}-\boldsymbol{p}_i\|-d_i$$

对 $\boldsymbol{x}$ 的 Hessian。

(3) 自己实现完整 Newton 方法。

(4) 使用同一个初始位置，分别运行：

(a) Newton；

(b) Gauss–Newton。

(5) 记录每轮：

$$\|H_{\text{Newton}} - H_{\text{GN}}\|_F.$$

(6) 绘制该误差随迭代次数的变化。

(7) 验证当 residual 越来越小时：

$$H_{\text{Newton}} \approx H_{\text{GN}}.$$

5 Levenberg–Marquardt

5.1 题目 6：实现完整 LM

继续使用 UWB 定位问题。

实现：

$$(J^T J + \lambda I)\Delta \boldsymbol{x} = -J^T \boldsymbol{r}.$$

定义：

$$\Delta F_{\text{actual}} = F(\boldsymbol{x}) - F(\boldsymbol{x} + \Delta \boldsymbol{x}),$$

以及：

$$\Delta F_{\text{pred}} = \frac{1}{2}\Delta \boldsymbol{x}^T(\lambda \Delta \boldsymbol{x} - \boldsymbol{g}).$$

定义 gain ratio：

$$\rho = \frac{\Delta F_{\text{actual}}}{\Delta F_{\text{pred}}}.$$

完成：

(1) 实现 LM，并根据 ρ 动态调整 λ。

(2) 采用如下规则：

$$\rho > 0$$

时接受 step。

若接受：

$$\lambda \leftarrow \lambda \max\left(\frac{1}{3}, 1-(2\rho-1)^3\right).$$

若：

$$\rho \le 0,$$

拒绝 step，并增加 λ。

(3) 记录每轮：

$$F_k, \quad \lambda_k, \quad \rho_k, \quad \|\Delta \boldsymbol{x}_k\|.$$

(4) 绘制：

(a) LM 优化轨迹；

(b) cost–iteration；

(c) λ–iteration；

(d) ρ–iteration。

(5) 与 Gauss–Newton 使用相同初值比较。

5.2 题目 7：研究 λ 对 LM 更新方向的影响

固定一个非最优位置：

$$\boldsymbol{x}_0.$$

计算：

$$H = J^T J, \qquad \boldsymbol{g} = J^T \boldsymbol{r}.$$

分别取：

$$\lambda = 0, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100, 1000.$$

对于每一个 λ 计算：

$$\Delta \boldsymbol{x}(\lambda) = -(H+\lambda I)^{-1}\boldsymbol{g}.$$

完成：

(1) 对所有方向进行归一化。

(2) 在同一张 cost 等高线上使用箭头绘制所有更新方向。

(3) 同时绘制负梯度方向：

$$-\nabla F.$$

(4) 计算角度：

$$\theta(\lambda) = \angle\left(\Delta \boldsymbol{x}(\lambda), -\boldsymbol{g}\right).$$

(5) 绘制：

$$\theta \quad \text{vs.} \quad \lambda.$$

(6) 从特征值分解

$$H = Q\Lambda Q^T$$

出发解释为什么：

$$\lambda \to \infty$$

时 LM 越来越接近梯度下降。

6 Line Search

6.1 题目 8：实现 Backtracking Line Search

考虑 Rosenbrock 函数：

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

其全局最优解为：

$$(x^*, y^*) = (1, 1).$$

完成：

(1) 推导梯度

$$\nabla f(x, y).$$

(2) 自己实现 Gradient Descent。

(3) 第一种方法使用固定步长：

$$\alpha = 10^{-3}.$$

(4) 第二种方法使用 Backtracking Line Search。

初始：

$$\alpha = 1.$$

使用 Armijo 条件：

$$f(\boldsymbol{x} + \alpha \boldsymbol{p}) \leq f(\boldsymbol{x}) + c_1 \alpha \nabla f^T \boldsymbol{p}.$$

推荐：

$$c_1 = 10^{-4}, \qquad \rho = 0.5.$$

(5) 初值：

$$\boldsymbol{x}_0 = (-1.5, 1.5).$$

(6) 绘制 Rosenbrock 函数的二维等高线。

(7) 将两种算法的迭代轨迹叠加在等高线上。

(8) 比较：

(a) 迭代次数；

(b) 最终误差；

(c) cost 下降速度。

6.2 题目 9：Gauss–Newton + Line Search

修改题目 4 的 UWB Gauss–Newton。

GN 原始方向：

$$J^T J \boldsymbol{p} = -J^T \boldsymbol{r}.$$

但不直接使用：

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{p}.$$

而改为：

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha \boldsymbol{p}.$$

通过 Backtracking Line Search 确定 α。

完成：

(1) 实现 GN + Armijo Line Search。

(2) 将初始位置设置为距离 anchor 区域较远的位置，例如：

$$\boldsymbol{x}_0 = (50, -30).$$

(3) 比较：

(a) 原始 GN；

(b) GN + Line Search。

(4) 记录每轮步长 α_k。

(5) 绘制两种方法的优化轨迹。

(6) 分析 Line Search 如何提高 GN 的全局收敛稳定性。

7 Trust Region

7.1 题目 10：二维 Trust Region 实验

仍使用 Rosenbrock 函数：

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

构造二次模型：

$$m_k(\boldsymbol{p}) = f_k + \boldsymbol{g}_k^T \boldsymbol{p} + \frac{1}{2} \boldsymbol{p}^T B_k \boldsymbol{p}.$$

使用约束：

$$\|\boldsymbol{p}\| \leq \Delta_k.$$

本题允许使用简化版 Trust Region：

(1) 计算 Newton step：

$$\boldsymbol{p}_N = -B_k^{-1} \boldsymbol{g}_k.$$

(2) 如果

$$\|\boldsymbol{p}_N\| \leq \Delta_k,$$

使用 Newton step。

(3) 否则使用沿负梯度方向、长度为 Δ_k 的 step：

$$\boldsymbol{p} = -\Delta_k \frac{\boldsymbol{g}_k}{\|\boldsymbol{g}_k\|}.$$

(4) 定义：

$$\rho_k = \frac{f(\boldsymbol{x}_k) - f(\boldsymbol{x}_k + \boldsymbol{p}_k)}{m_k(0) - m_k(\boldsymbol{p}_k)}.$$

(5) 根据以下规则更新：

$$\rho_k < 0.25 \Rightarrow \Delta_{k+1} = 0.25\Delta_k,$$

$$\rho_k > 0.75 \text{ 且} \|\boldsymbol{p}_k\| \approx \Delta_k \Rightarrow \Delta_{k+1} = 2\Delta_k.$$

(6) 若：

$$\rho_k > 0.1,$$

接受更新，否则拒绝。

(7) 绘制：

(a) 优化轨迹；

(b) Δ_k 随迭代次数变化；

(c) ρ_k 随迭代次数变化。

8 BFGS

8.1 题目 11：从零实现 BFGS

继续使用 Rosenbrock 函数。

BFGS 使用：

$$\boldsymbol{p}_k = -H_k \boldsymbol{g}_k,$$

其中 H_k 为 Hessian inverse 的近似。

定义：

$$\boldsymbol{s}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k,$$

$$\boldsymbol{y}_k = \boldsymbol{g}_{k+1} - \boldsymbol{g}_k,$$

以及：

$$\rho_k = \frac{1}{\boldsymbol{y}_k^T \boldsymbol{s}_k}.$$

逆 Hessian 更新：

$$H_{k+1} = (I - \rho_k \boldsymbol{s}_k \boldsymbol{y}_k^T) H_k (I - \rho_k \boldsymbol{y}_k \boldsymbol{s}_k^T) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^T.$$

完成：

(1) 从零实现 BFGS。

(2) 初始化：

$$H_0 = I.$$

(3) 使用 Backtracking Line Search 或 Wolfe Line Search。

(4) 初始位置：

$$\boldsymbol{x}_0 = (-1.5, 1.5).$$

(5) 绘制 BFGS 在 Rosenbrock 等高线上的迭代轨迹。

(6) 与 Gradient Descent 和 Newton 方法比较。

(7) 每轮计算：

$$\boldsymbol{y}_k^T \boldsymbol{s}_k.$$

(8) 检查该值是否保持正值。

(9) 分析：

$$\boldsymbol{y}_k^T \boldsymbol{s}_k > 0$$

与 BFGS 正定性的关系。

9 L-BFGS

9.1 题目 12：实现 Two-loop Recursion

实现 L-BFGS 的 two-loop recursion。

保存最近 m 组：

$$(\boldsymbol{s}_i, \boldsymbol{y}_i).$$

第一轮：

$$\boldsymbol{q} = \boldsymbol{g}_k,$$

从最新到最旧：

$$\rho_i = \frac{1}{\boldsymbol{y}_i^T \boldsymbol{s}_i},$$

$$\alpha_i = \rho_i \boldsymbol{s}_i^T \boldsymbol{q},$$

$$\boldsymbol{q} \leftarrow \boldsymbol{q} - \alpha_i \boldsymbol{y}_i.$$

初始逆 Hessian：

$$H_k^{(0)} = \gamma_k I,$$

其中：

$$\gamma_k = \frac{\boldsymbol{s}_{k-1}^T \boldsymbol{y}_{k-1}}{\boldsymbol{y}_{k-1}^T \boldsymbol{y}_{k-1}}.$$

令：

$$\boldsymbol{r} = H_k^{(0)} \boldsymbol{q}.$$

第二轮，从最旧到最新：

$$\beta_i = \rho_i \boldsymbol{y}_i^T \boldsymbol{r},$$

$$\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{s}_i(\alpha_i - \beta_i).$$

最终：

$$\boldsymbol{p}_k = -\boldsymbol{r}.$$

完成：

(1) 实现 two-loop recursion。

(2) 选择：

$$m = 3, 5, 10, 20.$$

(3) 使用 Rosenbrock 函数测试。

(4) 与完整 BFGS 比较：

(a) 迭代次数；

(b) 最终误差；

(c) 运行时间；

(d) 内存使用量。

(5) 分析历史长度 m 对 L-BFGS 性能的影响。

10 机器人轨迹估计综合实验

10.1 题目 13：二维机器人轨迹平滑

假设机器人沿二维平面运动，共有：

$$N = 50$$

个位姿位置变量：

$$\boldsymbol{x}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}.$$

真实轨迹可以自行构造，例如：

$$x_i = 0.2i,$$

$$y_i = 2\sin(0.2x_i).$$

加入位置测量噪声：

$$\boldsymbol{z}_i = \boldsymbol{x}_i^{\text{true}} + \boldsymbol{\epsilon}_i.$$

定义测量残差：

$$\boldsymbol{r}_i^{\text{obs}} = \boldsymbol{x}_i - \boldsymbol{z}_i.$$

为了使轨迹平滑，引入二阶平滑残差：

$$\boldsymbol{r}_i^{\text{smooth}} = \boldsymbol{x}_{i+1} - 2\boldsymbol{x}_i + \boldsymbol{x}_{i-1}.$$

构造目标函数：

$$F = \sum_i \|\boldsymbol{r}_i^{\text{obs}}\|^2 + \lambda_s \sum_i \|\boldsymbol{r}_i^{\text{smooth}}\|^2.$$

完成：

(1) 将所有状态拼接为：

$$\boldsymbol{X} = [x_1, y_1, x_2, y_2, \ldots, x_N, y_N]^T.$$

(2) 推导目标函数对应的 Jacobian 或直接构造线性最小二乘系统。

(3) 求解最优平滑轨迹。

(4) 比较：

$$\lambda_s = 0, 0.1, 1, 10, 100.$$

(5) 绘制：

   (a) 真值轨迹；

   (b) 带噪测量；

   (c) 优化后的轨迹。

(6) 分析 λ_s 太小和太大分别会发生什么。

11 可观测性与退化

11.1 题目 14：UWB 几何退化实验

二维 UWB 定位状态：

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \end{bmatrix}.$$

分别构造以下基站布局：

$$\text{布局 A：} \quad (0,0),(10,0),(10,10),(0,10),$$

$$\text{布局 B：} \quad (0,0),(3,0),(6,0),(9,0),$$

$$\text{布局 C：} \quad (0,0),(0.1,0),(0.2,0),(10,10).$$

完成：

(1) 对同一个机器人位置计算 Jacobian J。

(2) 对 J 做 SVD：

$$J = U\Sigma V^T.$$

(3) 输出所有奇异值。

(4) 计算条件数：

$$\kappa(J) = \frac{\sigma_{\max}}{\sigma_{\min}}.$$

(5) 绘制三种布局下的 UWB 基站和机器人位置。

(6) 比较三种布局下 GN 定位对测量噪声的敏感程度。

(7) 对每种布局执行 1000 次 Monte Carlo 仿真。

(8) 绘制定位误差分布。

(9) 解释：

$$\sigma_{\min} \to 0$$

为什么意味着某个状态方向越来越难以观测。

12 综合大作业

12.1 题目 15：实现一个二维机器人非线性优化实验框架

自行设计一个二维机器人估计问题，例如：

- UWB 定位；
- 多 landmark 距离定位；
- 二维激光 landmark 定位；
- 机器人轨迹平滑；
- 二维 pose graph；
- 非线性传感器融合。

要求至少实现以下四种算法：

1. Gauss–Newton；
2. Levenberg–Marquardt；
3. BFGS；
4. 一种带 Line Search 或 Trust Region 的方法。

程序应能够记录：

$$F_k, \qquad \|\nabla F_k\|, \qquad \|\Delta \boldsymbol{x}_k\|, \qquad \text{iteration}.$$

LM 还应记录：

$$\lambda_k, \qquad \rho_k.$$

要求完成以下实验：

(1) 使用完全相同的初值运行不同算法。

(2) 比较不同算法：

(a) 收敛速度；

(b) 迭代次数；

(c) 最终 cost；

(d) 最终状态误差；

(e) 单轮计算时间。

(3) 测试不同初值。

(4) 测试不同噪声水平。

(5) 人为构造一个较差初值，观察 GN 是否出现明显的大步更新。

(6) 分析 LM、Line Search 或 Trust Region 如何改善上述情况。

(7) 绘制优化过程动画。

(8) 绘制 cost surface 或 cost contour（若状态维度允许）。

(9) 在报告结尾讨论：

为什么不存在一种优化算法在所有机器人问题中都最好?

13 提高题

13.1 题目 16：加权 UWB 最小二乘

假设四个 UWB anchor 的测距标准差分别为：

$$\sigma_1 = 0.05, \qquad \sigma_2 = 0.1, \qquad \sigma_3 = 0.5, \qquad \sigma_4 = 1.0.$$

定义：

$$W = \mathrm{diag}\left(\frac{1}{\sigma_1^2}, \frac{1}{\sigma_2^2}, \frac{1}{\sigma_3^2}, \frac{1}{\sigma_4^2}\right).$$

加权目标函数：

$$F(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{r}^T W \boldsymbol{r}.$$

完成：

(1) 推导加权 Gauss–Newton：

$$J^T W J \Delta \boldsymbol{x} = -J^T W \boldsymbol{r}.$$

(2) 实现普通 GN 和加权 GN。

(3) 进行至少 500 次 Monte Carlo 实验。

(4) 比较两种方法的平均定位误差。

(5) 分析为什么低精度传感器应该得到较小权重。

13.2 题目 17：异常值与鲁棒核函数

在 UWB 测距中随机选择一个 anchor，并人为加入：

$$3 \sim 5 \text{ m}$$

的大异常值。

首先使用普通最小二乘：

$$F = \frac{1}{2} \sum_i r_i^2.$$

再使用 Huber loss：

$$\rho(r) = \begin{cases} \frac{1}{2} r^2, & |r| \le \delta, \\ \delta \left(|r| - \frac{1}{2} \delta \right), & |r| > \delta. \end{cases}$$

完成：

(1) 实现普通 GN。

(2) 实现基于 Huber kernel 的迭代加权最小二乘。

(3) 比较有无异常值情况下的定位结果。

(4) 绘制 residual 大小与对应权重之间的关系。

(5) 分析为什么平方损失对 outlier 十分敏感。

14 建议评分

建议总分为 100 分。

内容	比例
数学建模与公式推导	20%
算法独立实现	30%
代码正确性与数值稳定性	15%
可视化与实验设计	15%
实验结果分析	15%
代码规范与报告质量	5%

15 建议完成顺序

推荐按照以下顺序完成：

$$\boxed{1 \to 2 \to 3 \to 4 \to 6 \to 7 \to 8 \to 9 \to 10 \to 11 \to 12 \to 14 \to 15}$$

其中：

- 题目 1–4 用于建立最小二乘基本能力；
- 题目 5–7 用于理解 Newton、GN、LM；
- 题目 8–10 用于理解优化的 globalization strategy；
- 题目 11–12 用于理解 Quasi-Newton；
- 题目 13–17 开始接近实际机器人优化问题。